# Trainable Classifiers — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

# Contents


Executive Summary — Trainable Classifiers in Microsoft Purview 4

The problem . . . 4

The solution . . . 4

How it works — the model . . . 4

What you get . . . 4

Scope and boundaries . . . 5

The bottom line . . . 5

Solution Overview — Trainable Classifiers in Microsoft Purview 5

What this solution delivers . . . 5

The classifier lifecycle (how it actually works) . . . 6

What trainable classifiers are (and are not) . . . 6

Simulate first — auto-labeling and DLP are high-impact . . . 7

The standard deployment model . . . 7

What's in the box . . . 7

Requirements — Trainable Classifiers in Microsoft Purview 8

R1. Licensing for trainable classifiers and the controls they feed . . . 8

R2. Least-privilege roles for classifiers + the target control . . . 8

R3. Seed content — a SharePoint site of positive example documents (for custom classifiers) . . . 9

R4. Test content — a separate, pre-categorized set to measure accuracy . . . 9

R5. The target control decided: auto-labeling and/or DLP and/or retention . . . 9

R6. Built-in classifier catalog reviewed (use ready-made where possible) . . . 10

R7. An accuracy-acceptance decision — agreed with the data owners . . . 10

R8. A named pilot scope and change control (simulation/test first) . . . 10

Global Requirements — Microsoft Purview Solutions 10

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11

2. Purview roles (least-privilege) and portal access . . . 11

3. Security & Compliance PowerShell + Graph toolchain . . . 11

4. Data sources & scope in scope . . . 11

5. Compliance, records, privacy & legal stakeholders . . . 12

6. A pilot scope and change control . . . 12

Implementation Guide — Trainable Classifiers in Microsoft Purview 12

Phase 0 — Assess (confirm you are ready) . . . 13

Phase 1 — Design (built-in vs custom, target label/action) . . . 14

Phase 2 — Build the classifier (portal): seed → train → test → publish . . . 14

Phase 3 — Wire the classifier into a control, in SIMULATION . . . 14

Phase 4 — Pilot & validate in simulation . . . 15
Phase 5 — Enforce, then expand . . . 15
Rollback summary (fast reference) . . . 16
Testing & Validation — Trainable Classifiers 16
A. Classifier accuracy gates (manual — in the portal Test stage + simulation) . . . 17
B. Wiring configuration gates (automated — `CLSTRAIN-Validate.ps1`, read-only) . . . 17
C. Behavioural tests — does the wired control act on classified content? (both passes) . . . 18
D. Evidence to capture (for the defensibility pack) . . . 18
Troubleshooting — Trainable Classifiers 19
1. "I published the classifier but nothing happens" . . . 19
2. The classifier won't train / stays "training" . . . 19
3. Low accuracy at the Test stage . . . 20
4. The classifier matches the wrong content in the real world (false positives) . . . 20
5. A built-in classifier isn't available / behaves differently than expected . . . 20
6. `New-AutoSensitivityLabelPolicy` / label errors . . . 21
7. Can't reference the classifier as a rule condition in PowerShell . . . 21
8. Simulation looks fine but enforcement doesn't label . . . 21
9. Wrongly-applied labels need reversing . . . 22
10. Activity report is empty or export fails . . . 22
When to escalate to the determination owners (not a technical fix) . . . 22
Best Practices — Trainable Classifiers 22
Use built-in classifiers before you build custom ones . . . 23
Seed and test like a data scientist . . . 23
Never publish or enforce a classifier that fails its accuracy gate . . . 23
The classifier is only half the job — wire it, in simulation . . . 23
Mind the high-impact, hard-to-reverse nature of labeling . . . 23
Operate it, don't set-and-forget . . . 24
Keep the roles and determinations where they belong . . . 24
Security & handling of the tooling . . . 24
Deliverables Summary — Trainable Classifiers in Microsoft Purview 24
Documents . . . 25
PowerShell tools (5) . . . 25
How it maps to the deployment model . . . 26
Definition of done . . . 27
Appendix A — PowerShell Scripts 28

# Executive Summary — Trainable Classifiers in Microsoft Purview

## The problem

Some of an organization's most sensitive content can't be found by keywords or patterns. A **contract**, a **résumé**, a **source-code** file, a **statement of work** — these are recognized by their *shape and meaning*, not a fixed string, so pattern-based sensitive information types miss them entirely. That leaves a whole class of important documents unlabeled, un-governed, and unprotected — both a **compliance gap** and a **data-risk exposure**.

## The solution

This solution delivers **Microsoft Purview trainable classifiers** end to end — machine-learning classifiers that recognize content **by example** rather than by pattern — and wires them into the controls that act on classification:

- **Built-in classifiers** (e.g. Resumes, Source Code) work immediately, with no training.
- **Custom classifiers** are **seeded** with example documents, **trained**, **tested for accuracy**

against a separate known set, and **published** only when accuracy is acceptable.

- The published classifier is **wired into an auto-labeling policy** (and can feed DLP and retention)

so matching content is automatically labeled and protected — **deployed in simulation first**, validated, then enforced.

The critical discipline: a classifier is **tested for accuracy before it is published**, and the control it feeds is **piloted in simulation** — it records what it *would* label without changing anything — before any enforcement.

## How it works — the model

The delivery follows a disciplined, repeatable model with an explicit pass/fail gate at each step:

**Assess** → **Design** → **Configure (seed/train/test/publish + wire)** → **Pilot (simulation)** → **Validate** → **Expand (enforce).**

Because auto-labeling can apply **encrypting** labels and DLP can block, nothing is enforced until the classifier has passed an **accuracy gate** — on both a curated test set and a real-world simulation — and the data owner has signed off.

## What you get

- A complete document set: Overview, Requirements, an ordered Implementation guide, Testing (accuracy

+ wiring pass/fail gates), Troubleshooting, Best Practices, and this summary.

- **Five PowerShell tools** (Security & Compliance PowerShell / Microsoft Graph) — discovery,

configuration of the **wiring** (simulation, `-WhatIf` first), validation, reporting, and scope/health — all **read-only or preview by default, with no credentials in code.** (Classifier creation itself is a portal action; the tools automate the wiring and reporting around it.)

- A **defensibility evidence pack**: measured accuracy, proof the classifier was published only after passing its threshold, that the wiring was validated in simulation, and that the label/action applies only to the right content.

## Scope and boundaries

- Trainable classifiers and the controls they feed (auto-labeling, advanced DLP, retention) are **licensing-dependent** (typically Microsoft 365 E5 / E5 Compliance / E5 Information Protection & Governance) and the Purview portal and built-in catalog **evolve** — capability availability, seed minimums, and exact paths are **verified against your licensing/tenant**, never assumed.
- **Regulatory, records, and legal determinations** — which label a category deserves, what accuracy is acceptable, and whether content is in scope — remain with your **compliance, records, privacy, and data-owner** functions. This solution provides the **technical method, the accuracy evidence, and the wiring**, not the determination.

## The bottom line

Content that keyword and pattern rules can't find becomes **classified by meaning** and automatically labeled/protected in Microsoft Purview — **seeded, trained, tested, wired, piloted in simulation, validated, and defensible.**

# Solution Overview — Trainable Classifiers in Microsoft Purview

## What this solution delivers

Some content can't be found by keywords or patterns. A **contract** doesn't announce itself with a regular expression; a **résumé**, a **source-code** file, or a **statement of work** is recognized by its *shape and meaning*, not a fixed string. Pattern-based sensitive information types (SITs) miss this whole class of data — and that leaves it unlabeled, un-governed, and unprotected. Left unaddressed, that is both a **compliance gap** and a **data-risk exposure**.

This solution delivers **Microsoft Purview trainable classifiers** end to end — machine-learning classifiers that recognize content **by example** rather than by pattern — and, crucially, **wires them into the controls that act on classification**:

1. **Built-in (pre-trained) classifiers.** Microsoft ships ready-to-use classifiers (e.g. **Resumes**, **Source Code**, and others — **verify the current catalog for your tenant**) that work immediately with no training.
1. **Custom trainable classifiers.** For content specific to your organization, you **seed** the

classifier with example documents, let it **train**, **test** it against a separate known set to measure accuracy, and **publish** it.

1. **Wiring to labels, DLP, and retention.** A classifier is only useful once something *acts* on it.

This solution wires the published classifier into an **auto-labeling** policy (and shows how it is referenced by **DLP** and **retention**) so matching content is automatically labeled/protected/ retained — deployed in **simulation** first, validated, then enforced.

The result is *classify content by meaning, not keywords* — seeded, trained, tested, wired, **piloted in simulation**, validated, and defensible.

## The classifier lifecycle (how it actually works)

A custom trainable classifier follows a defined lifecycle in the Purview portal (**Data classification** → **Classifiers** → **Trainable classifiers** — *verify path for your tenant*):

| Stage | What you do | Outcome |
|---|---|---|
| **Seed** | Point the classifier at a **SharePoint site of positive example documents** (Microsoft recommends a meaningful minimum number of samples — **verify the current minimum/recommended counts**). | The classifier learns what "positive" looks like. |
| **Train** | The service processes the seed content. | A candidate model. |
| **Test** | Provide a **separate test set** of items you have **pre-labeled** as match / non-match; review the classifier's predictions and its accuracy. | A measured model you can trust (or retrain). |
| **Publish** | Publish the classifier once accuracy is acceptable. | The classifier becomes **available as a condition** in auto-labeling, DLP, and retention. |
| **Wire & use** | Reference the classifier in an **auto-labeling / DLP / retention** policy — **in simulation first.** | Content is automatically labeled / protected / retained by *meaning*. |

> ***The classifier is only half the job.*** *Publishing a classifier changes nothing on its own — it must be* ***wired into a control*** *(auto-labeling, DLP, or retention) to have an effect. This solution treats the wiring as a first-class deliverable, not an afterthought.*

## What trainable classifiers are (and are not)

- **They are** ML classifiers that recognize content **by example/category** — good for the

"recognizable by shape, not by string" content (contracts, resumes, source code, SOWs, financial statements) that SITs and keyword lists cannot reliably catch.

• **They are not** a replacement for **sensitive information types** or **exact data match**. Use SITs for well-structured identifiers (card numbers, national IDs) and classifiers for *categories of document*. They complement each other.

• **They are licensing-dependent.** Trainable classifiers and the controls they feed (auto-labeling, advanced DLP, retention) are premium capabilities (typically **Microsoft 365 E5** / **E5 Compliance** / **E5 Information Protection & Governance**) — **verify capability availability and the built-in classifier catalog against your licensing/tenant**, and note the portal evolves (some classifiers/settings are newer or in preview). See `global-requirements-purview.md` §1.

## Simulate first — auto-labeling and DLP are high-impact

Auto-applying a **sensitivity label** (which may carry **encryption**) or a **DLP block** to content a classifier matches is high-impact — an over-eager classifier could label or restrict the wrong documents. Therefore:

• The classifier is **tested for accuracy before publishing**, and the control it feeds (auto-labeling / DLP) is deployed in **simulation/test mode first** — it **records what it *would* label/act on without changing anything** — validated on a pilot scope, and only then enforced.

> ***Determinations are not yours to make.*** **Which* label a category should receive, *whether* the classifier's accuracy is acceptable for a given use, and *whether* content is in scope are* ***compliance / records / data-owner*** *determinations — this solution delivers the technical method, the accuracy evidence, and the wiring, not the determination (see* `global-requirements-purview.md` *§5).*

## The standard deployment model

Every step follows the same repeatable model, with an explicit **pass/fail gate** between phases:

**Assess** → **Design** → **Configure (seed/train/test/publish + wire)** → **Pilot (simulation)** → **Validate** → **Expand (enforce).**

• **Assess** — confirm licensing, least-privilege roles, the seed/test content, the target label/DLP/retention control, the pilot scope, and stakeholder sign-off.

• **Design** — decide which built-in classifiers to use, which custom categories to build, the target label/action for each, and name every object.

• **Configure** — seed → train → **test (measure accuracy)** → publish the classifier(s), then **wire** into an auto-labeling (and/or DLP/retention) policy **in simulation**.

• **Pilot** — run the wired control in simulation; confirm matching content is *classified* and the control *would* act.

• **Validate** — run the pass/fail gates in `05-testing.md` (accuracy + wiring).

• **Expand** — publish/enforce and widen scope **only** after the gate passes; roll back per `03-implementation.md` if a gate fails.

## What's in the box

• This overview, a **Requirements** guide, an ordered **Implementation** guide, **Testing**

(accuracy + wiring pass/fail gates), **Troubleshooting**, **Best Practices**, and a **Deliverables Summary** + **Executive Summary**.

• **Five PowerShell tools** (Security & Compliance PowerShell / Microsoft Graph): a **Discovery**

inventory of where classifiers can be wired (auto-labeling, DLP, retention, labels); a **Configure/Remediation** helper that builds the **wiring** (auto-labeling policy) in **simulation (**`-WhatIf` **first)**; a **Validation** checker; a **Reporting** rollup of policy modes and classification activity; and a **seed/scope health** helper — **read-only or** `-WhatIf` **by default; no credentials in code.**

> ***Classifier creation is portal-driven.*** *Seeding, training, testing, and publishing a trainable classifier are done in the* ***Purview portal*** *— the scripts do* ***not*** *invent a cmdlet to create or publish a classifier; they automate the* ***wiring*** *(auto-labeling/DLP/retention) and the reporting around it, and remind you to confirm classifier status in the portal.*
>
> ***Sources.*** *Microsoft Learn — *Learn about trainable classifiers*, *Create a trainable classifier*, *Get started with trainable classifiers*, *Apply a sensitivity label to content automatically* (auto-labeling + simulation), and *Create a DLP policy / retention using a trainable classifier as a condition*. These capabilities are* ***licensing-dependent*** *and the Purview portal is* ***evolving*** *—* ***verify capability availability, the built-in catalog, minimum seed counts, portal paths, and cmdlet parameters against your licensing/tenant*** *rather than assuming.*

# Requirements — Trainable Classifiers in Microsoft Purview

Confirm every prerequisite below **before** you configure anything. Category-level prerequisites (tenant, compliance licensing model, least-privilege roles, Security & Compliance PowerShell / Graph toolchain, in-scope data sources, compliance/privacy/legal stakeholders, pilot scope + change control) are in `global-requirements-purview.md` — reference them; they are **not** repeated here. The items below are the ones **specific to trainable classifiers**: the **seed/test content**, the **target control** (label/DLP/retention), and the **accuracy acceptance** decision.

Each requirement is written as **Description · Why · Verify · Implement if missing.**

## R1. Licensing for trainable classifiers and the controls they feed

**Description:** The licensing that includes **trainable classifiers** and the downstream control you will wire them into — **auto-labeling**, **advanced DLP**, and/or **retention** — typically **Microsoft 365 E5**, the **E5 Compliance**, or the **E5 Information Protection & Governance** add-on, assigned to the in-scope users/scope. **Why:** Trainable classifiers and auto-labeling/advanced DLP are premium, plan-gated capabilities; without the right licence they are unavailable, and content-level enforcement is licensed per user. **Verify:** Microsoft 365 admin center → **Billing** → **Licenses** confirms the SKU/add-on. In the Purview portal, **Data classification** → **Classifiers** → **Trainable classifiers** is present, and the target control (auto-labeling / DLP / retention) is available. Cross-check against the *Microsoft Purview service description* — **verify against your licensing/tenant. Implement if missing:** Acquire and assign the required SKU/add-on before building classifiers or wiring controls.

## R2. Least-privilege roles for classifiers + the target control

**Description:** The least-privilege roles, mapped to **named** people:

• The rights to **create/manage trainable classifiers** (typically a data-classification / information-

protection role — **verify the exact role in your tenant**).

• **Compliance Administrator / Compliance Data Administrator / Information Protection** roles to author

the **auto-labeling / DLP / retention** policy the classifier feeds.

• **Global Administrator** only where genuinely required.

**Why:** Purview uses fine-grained role groups; classifier management and policy authoring are distinct duties and should not be over-granted. **Verify:** Purview portal → **Roles & scopes / Permissions** confirms the memberships. `CLSTRAIN-Discovery.ps1` reports the policy/label signals it can read. **Implement if missing:** Assign the least-privilege role for each duty under change control.

## R3. Seed content — a SharePoint site of positive example documents (for custom classifiers)

**Description:** For each **custom** classifier, a **SharePoint site (or location)** containing a set of **positive example documents** that represent the category (e.g. contracts), meeting Microsoft's **minimum/recommended sample counts** — **verify the current minimums for your tenant** (they are non-trivial; too few samples yields a weak model). **Why:** A trainable classifier learns **by example**; the quality and quantity of the seed set directly determines accuracy. Built-in classifiers need no seed content. **Verify:** Confirm the seed site exists and holds enough representative, correctly-categorized documents. `CLSTRAIN-ScopeHealth.ps1` prompts you to confirm the seed site and reports what it can read. **Implement if missing:** Assemble a representative seed set on a dedicated SharePoint site **before** creating the custom classifier. (Avoid mixing categories — seed = positive examples of *one* category.)

## R4. Test content — a separate, pre-categorized set to measure accuracy

**Description:** A **separate** set of items (not the seed set) that you have **pre-categorized** as **match / non-match**, used in the classifier's **Test** stage to measure prediction accuracy before publishing. **Why:** Publishing a classifier that has not been tested for accuracy risks mislabeling/mis-acting on content at scale. The test stage is where you decide whether accuracy is acceptable (R7). **Verify:** Confirm a distinct test set exists with a known ground truth (which items *should* match). **Implement if missing:** Assemble a labeled test set (mix of true matches and true non-matches) before the Test stage.

## R5. The target control decided: auto-labeling and/or DLP and/or retention

**Description:** A decision on **what acts on the classifier** — an **auto-labeling** policy (apply a sensitivity label), a **DLP** policy (protect/restrict), and/or a **retention** policy (retain/dispose) — and, for auto-labeling, **which sensitivity label** the matched content should receive. **Why:** A published classifier does nothing until it is **wired into a control**; the target control and (for labeling) the specific label are the point of the whole exercise. **Verify:** Purview → **Information Protection** → **Labels** confirms the target label exists and is published; the target control (auto-labeling/DLP/retention) is available. `CLSTRAIN-Discovery.ps1` inventories labels and the candidate policies. **Implement if missing:** Confirm/publish the target label and decide the target control **before** wiring. (A label taxonomy rollout may be a separate project; classifiers can start by feeding an audit-only DLP or a simulation auto-label policy.)

### R6. Built-in classifier catalog reviewed (use ready-made where possible)

**Description:** A review of the **built-in (pre-trained) classifiers** available in your tenant (e.g. Resumes, Source Code, and others — **verify the current catalog**) to decide which categories are already covered without any training, versus which need a **custom** classifier. **Why:** Built-in classifiers work immediately with no seed/test effort; building custom classifiers for content Microsoft already covers is wasted effort. **Verify:** Purview → **Data classification** → **Classifiers** → **Trainable classifiers** shows the built-in classifiers. **Verify the catalog against your tenant** — it evolves. **Implement if missing:** Map each needed category to a built-in classifier where one exists; reserve custom classifiers for organization-specific content.

### R7. An accuracy-acceptance decision — agreed with the data owners

**Description:** A documented decision, agreed with the **data owners / compliance**, of **what accuracy is acceptable** for the classifier's intended use (a classifier feeding an audit-only report can tolerate more error than one auto-applying an encrypting label), and who signs off the Test-stage results before publishing. **Why:** "Is this classifier good enough?" is a **business/risk determination**, not a technical default — the tolerance depends on what the classifier will drive. **Verify:** Confirm the accuracy threshold and the named approver **in writing** before publishing and before moving the wired control from simulation to enforce. **Implement if missing:** Capture the acceptance criteria with the data owners; until then, keep the wired control in **simulation / audit only.** (See `global-requirements-purview.md` §5.)

### R8. A named pilot scope and change control (simulation/test first)

**Description:** A **named pilot scope** (users/sites, e.g. `SG-Classifier-Pilot`) for the wired control, and agreement to run the auto-labeling/DLP policy in **simulation/test mode first**, under a **change-control** process, before any enforcing, broad rollout. **Why:** Auto-labeling (which can encrypt) and DLP block actions are high-impact; simulation on a small scope proves the classifier + control behave as measured **without** changing content, and lets you tune before enforcing. **Verify:** Confirm the pilot scope exists, that **simulation/test mode** is used, and a change window/approval path. `CLSTRAIN-ScopeHealth.ps1` (`-WhatIf`) previews pilot-scope targeting. **Implement if missing:** Define the pilot scope, deploy the wired control in simulation, and agree change control before enabling enforcement.

## Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention,*

> ***and legal determinations*** *(retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are* ***deferred to the organization's compliance, records, privacy, and legal functions*** *— the solutions provide the technical configuration method and evidence, not the determination. Automation is* ***PowerShell*** *(Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

## 1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

## 2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

## 3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

• **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

## 4. Data sources & scope in scope

• **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

## 5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

## 6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

# Implementation Guide — Trainable Classifiers in Microsoft Purview

This is the ordered, do-it-in-this-sequence guide. It follows the standard model — **Assess** → **Design** → **Configure (seed/train/test/publish + wire)** → **Pilot (simulation)** → **Validate** → **Expand (enforce)** — with every object **named**, an explicit **pass/fail gate** at each phase, and a **rollback** for each change. Do **not** skip the accuracy-test or the simulation phase; auto-labeling and DLP acting on a classifier are high-impact.

**Named objects used throughout (rename to your convention, but be consistent):**

| Object | Name used here |
|---|---|
| Custom trainable classifier | `TC-Contracts` |
| Built-in classifier(s) referenced | e.g. `Resumes`, `Source Code` (verify catalog) |
| Seed SharePoint site | `Seed-Contracts` (positive examples of contracts) |
| Target sensitivity label | `Confidential` (or a sub-label, per your taxonomy) |
| Auto-labeling policy (the wiring) | `AL-Classifier-Pilot` |
| Auto-labeling rule | `AL-Rule-Contracts` |
| Pilot scope (users/sites) | `SG-Classifier-Pilot` |

> ***Tooling note. Creating, seeding, training, testing, and publishing a trainable classifier are done in the Purview portal*** *— there is* ***no*** *supported Security & Compliance PowerShell cmdlet to create/publish a classifier, so those steps are portal actions here. The* ***wiring*** *(auto-labeling / DLP / retention) can be authored in the portal* ***or*** *with* `New-/Set-AutoSensitivityLabelPolicy` *and* `New-/Set-AutoSensitivityLabelRule` *(and the equivalent DLP / retention cmdlets); the provided* `CLSTRAIN-Configure.ps1` *builds the auto-labeling wiring in* ***simulation*** *with* `-WhatIf` *by default.* ***Verify every portal path and cmdlet parameter — especially how a trainable classifier is referenced as a condition — against your tenant*** *(this evolves).*

## Phase 0 — Assess (confirm you are ready)

1. **Confirm requirements.** Walk `02-requirements.md` R1–R8. Do **not** proceed until licensing,

roles, seed content, test content, the target control + label, the built-in catalog review, the accuracy-acceptance decision, and a pilot scope are all confirmed.

1. **Run Discovery.** `Connect-IPPSSession`, then `.\CLSTRAIN-Discovery.ps1` to inventory existing

auto-labeling / DLP / retention policies, sensitivity labels, and where a classifier could be wired. Keep the CSV as your baseline.

1. **Confirm seed readiness.** `.\CLSTRAIN-ScopeHealth.ps1` prompts you to confirm the seed site and

reports pilot-scope signals.

**Gate 0 (pass/fail):** All of R1–R8 satisfied, Discovery baseline captured, seed + test content ready, pilot scope defined. **Fail** → **stop** and close the gap; do not configure.

## Phase 1 — Design (built-in vs custom, target label/action)

1. **Map categories to classifiers** (R6): for each category you need, decide **built-in** (use as-is) vs **custom** (`TC-Contracts`). Only build custom where no built-in covers it.

1. **Decide the target control + label** (R5): e.g. auto-apply the `Confidential` label to content matching `TC-Contracts`; or feed a **DLP** rule / **retention** policy.

1. **Name every object** per the table above and set the **pilot scope**.

**Gate 1:** The classifier map, target control/label, and named objects are documented and approved. **Fail** → **stop**; the accuracy tolerance (R7) and label choice are determination-owner calls.

## Phase 2 — Build the classifier (portal): seed → train → test → publish

For each **custom** classifier (built-in classifiers skip straight to Phase 3):

1. In **Purview portal** → **Data classification** → **Classifiers** → **Trainable classifiers** (verify path), **create** `TC-Contracts` and point its **seed** at the `Seed-Contracts` SharePoint site (R3).

1. Let it **train**. Training is not instant — allow the processing time Microsoft indicates.

2. **Test** the trained classifier against your **separate, pre-categorized test set** (R4). Review the predictions and the accuracy against the **R7 acceptance threshold**.

• If accuracy is **below** threshold: add/clean seed samples, retrain, retest. Do **not** publish a classifier that fails R7.

1. **Publish** the classifier **only when the R7 threshold is met** and the data owner has signed off.

**Gate 2:** `TC-Contracts` is **published** with test-stage accuracy at/above the **R7** threshold, signed off in writing. Built-in classifiers are confirmed available. **Fail** → **do not publish**; keep tuning the seed set. (Rollback: an unpublished classifier acts on nothing; delete/retrain it.)

## Phase 3 — Wire the classifier into a control, in SIMULATION

Create the wiring that **acts** on the classifier — here an **auto-labeling** policy (DLP / retention follow the same simulate-first pattern). Author in the portal or run `.\CLSTRAIN-Configure.ps1 -WhatIf` first, then `.\CLSTRAIN-Configure.ps1 -Commit` (still simulation).

1. **Create the auto-labeling policy** `AL-Classifier-Pilot` in **simulation** (portal: auto-labeling *simulation mode*; PowerShell: `New-AutoSensitivityLabelPolicy` with a **Test** mode such as `TestWithoutNotifications` — **verify the exact mode value in your tenant**), scoped to the pilot, applying the `Confidential` label.

1. **Add the rule** `AL-Rule-Contracts` with a **condition that references the trainable classifier** (`TC-Contracts`, and/or a built-in classifier). **Verify how a trainable classifier is referenced as a condition in** `Get-Help New-AutoSensitivityLabelRule -Full` **for your tenant** — the parameter shape evolves; if uncertain, attach the classifier condition **in the portal** and read it back with Discovery.

1. **Save.** The policy now evaluates in simulation — it shows **what it *would* label** without

applying anything.

**Gate 3:** `AL-Classifier-Pilot` exists, is in **simulation**, scoped to the pilot, applies the target label, with `AL-Rule-Contracts` referencing the classifier. `CLSTRAIN-Validate.ps1` reports **PASS** for policy-exists, mode-is-simulation, scope-is-pilot, rule-present, and classifier/label-referenced (best-effort). **Fail** → **rollback:** `Set-AutoSensitivityLabelPolicy -Identity AL-Classifier-Pilot -Mode Disable` (or delete with `Remove-AutoSensitivityLabelPolicy`); simulation changes nothing.

## Phase 4 — Pilot & validate in simulation

1. Let the auto-labeling policy run in simulation over the pilot scope for the observation window

(auto-labeling simulation processes existing content — allow time).

1. Review the **simulation results** (portal): which items the classifier matched and *would* have

labeled. Run `.\CLSTRAIN-Report.ps1` to roll up policy modes and (where available) classification activity.

1. **Assess real-world accuracy:** spot-check the simulation matches against ground truth — does the

classifier match the right documents in *your* estate (not just the test set)? Tune the seed/retrain (Phase 2) if the real-world precision disappoints.

**Gate 4 (pass/fail):** The classifier matched the intended content in simulation, real-world precision meets the **R7** threshold, false positives triaged, and the target label/action confirmed **in writing**. **Fail** → **do not enforce**; retrain/tune. Rollback unchanged from Phase 3.

## Phase 5 — Enforce, then expand

**Only after Gate 4 passes.**

1. **Move the auto-labeling policy out of simulation** to **enforce** (portal: turn the policy **On**;

PowerShell: `Set-AutoSensitivityLabelPolicy -Identity AL-Classifier-Pilot -Mode Enable`), still scoped to the pilot first. (For a DLP/retention wiring, enforce that policy instead, per its own simulate-first pattern.)

1. **Confirm** the label is now actually applied to matching pilot content (allow processing time).
2. **Expand scope in rings.** Widen the auto-labeling scope (add the next users/sites), re-validate,

and repeat. Consider renaming/cloning to a production policy (`AL-Classifier`) as you exit pilot.

1. **Reuse the classifier.** The same published classifier can now be referenced by **DLP** and

**retention** — each wired **in simulation first** and validated before enforce.

**Gate 5:** On the pilot, matching content is labeled/acted-on as designed and non-matching content is untouched; expansion rings each pass the same gate before widening. **Fail** → **rollback:** return the policy to **simulation** (`Set-AutoSensitivityLabelPolicy -Mode TestWithoutNotifications`) or **Disable** it — this stops auto-labeling at once — then diagnose per `06-troubleshooting.md` before

re-enforcing. (Note: labels already applied are not auto-removed — plan re-labeling/label-change under change control if you must reverse applied labels.)

### Rollback summary (fast reference)

| Situation | Action |
|---|---|
| The classifier fails accuracy (R7) | Do **not** publish/enforce; add/clean seed samples, retrain, retest in the portal. |
| The wired policy labels the wrong content | Set `AL-Classifier-Pilot` to **simulation/test**, or **Disable** it; retrain the classifier or tighten the rule. |
| The whole wiring misbehaves | `Set-AutoSensitivityLabelPolicy -Identity AL-Classifier-Pilot -Mode Disable` (stops auto-labeling) or `-Mode TestWithoutNotifications` (back to simulation). |
| A user/site should not be in scope | Remove it from the policy scope (`SG-Classifier-Pilot`). |
| Labels wrongly applied need reversing | Plan a controlled re-label (labels applied are not auto-removed by disabling the policy) under change control. |
| Start over cleanly | `Remove-AutoSensitivityLabelRule` then `Remove-AutoSensitivityLabelPolicy`; delete/retrain the classifier in the portal. |

> *Enforcement changes are **high-impact** — auto-labeling can apply **encrypting** labels. Make changes under change control, keep the pilot scope small, and never enforce a classifier that has not passed its accuracy gate (R7) in both the test set and the simulation pass.*

## Testing & Validation — Trainable Classifiers

This is the pass/fail gate set. Trainable classifiers have **two** things to validate: the **classifier's accuracy** (does it recognize the right content?) and the **wiring** (does the control that acts on it exist, in simulation, correctly scoped?). Run the wiring gates in two passes: **once in simulation** (Phase 4 — content is *classified* and the control *would* act, nothing changes) and **again after enforce** (Phase 5 — the label/action *actually* applies). Use **benign test content** with a known ground truth — **never** real regulated data as your only test material.

> ***Determination boundary.*** *These tests prove the **classifier and control work as configured** and measure accuracy. Whether the accuracy is *acceptable* for a given use, and whether content is in scope, are **compliance / data-owner** determinations — not test outcomes (see `global-requirements-purview.md` §5).*

Run `.\CLSTRAIN-Validate.ps1` for the wiring configuration gates and `.\CLSTRAIN-Report.ps1` for the policy-mode/activity rollup.

## A. Classifier accuracy gates (manual — in the portal Test stage + simulation)

| # | Check | PASS signal |
|---|---|---|
| A1 | Test-stage accuracy | In the classifier's **Test** stage, accuracy meets the **R7** threshold against the pre-categorized test set (R4). |
| A2 | True positives | Known-match test items are predicted **match**. |
| A3 | True negatives | Known non-match test items are predicted **non-match** (precision, not just recall). |
| A4 | Real-world precision | In the **simulation** pass over the pilot estate, spot-checked matches are genuinely the target category (not just the curated test set). |
| A5 | Published only if passed | The classifier was **published only after** A1–A4 met the R7 threshold, signed off by the data owner. |

**Gate A:** A1–A5 PASS at the R7 threshold. Any FAIL → retrain (add/clean seed samples) and retest; **do not publish or enforce** a classifier that fails accuracy.

## B. Wiring configuration gates (automated — `CLSTRAIN-Validate.ps1`, read-only)

| # | Check | PASS signal |
|---|---|---|
| B1 | Wiring policy exists | `AL-Classifier-Pilot` (auto-labeling) is present. |
| B2 | Simulation first | In the pilot pass, the policy **Mode** is a **Test/simulation** mode (not `Enable`). |
| B3 | Pilot scope | The policy is scoped to `SG-Classifier-Pilot`, **not** all users/sites. |
| B4 | Rule present | `AL-Rule-Contracts` exists and is enabled. |
| B5 | Target label set | The policy applies the intended sensitivity label (e.g. `Confidential`). |

| # | Check | PASS signal |
|---|---|---|
| B6 | Classifier referenced | The rule references the trainable/built-in classifier as a condition (best-effort; **confirm in the portal where PowerShell does not expose the classifier condition**). |

**Gate B:** All applicable rows PASS. Any FAIL → fix in `03-implementation.md` before the behavioural pass.

## C. Behavioural tests — does the wired control act on classified content? (both passes)

For each, place/identify a **matching** test document and a **non-matching** one in the pilot scope and record the result. Simulation PASS = *"item shown in simulation results as would-be-labeled; nothing applied."* Enforce PASS = *"the label is actually applied."*

| # | Test | Simulation PASS | Enforce PASS |
|---|---|---|---|
| C1 | A **matching** document (target category) in the pilot scope | Appears in simulation results as *would apply* the label | The target label is **applied** to it |
| C2 | A **non-matching** document in the pilot scope | **Not** matched; would not be labeled | **Not** labeled |
| C3 | A matching document **outside** the pilot scope | Not processed by this policy | Not labeled by this policy |

**Gate C (simulation pass):** C1 matches and C2 does not — the classifier + wiring discriminate correctly. **Gate C (enforce pass):** C1's label is applied and C2 is untouched. Any deviation → roll back to simulation and retrain/tune.

## D. Evidence to capture (for the defensibility pack)

Capture and retain:

• The classifier **Test-stage accuracy** results and the **R7 sign-off** (accuracy threshold + named approver).

• `CLSTRAIN-Validate.ps1` output (wiring configuration gates).

• `CLSTRAIN-Report.ps1` output (policy modes + classification activity export where available) and the **auto-labeling simulation results** (which items would be / were labeled).

• Screenshots of the classifier's published status, the simulation results, and an applied label in

the enforce pass.

- The **classifier map** (built-in vs custom, target label/action) and the pilot-scope confirmation.

*This evidence set is what makes the classifier **defensible** — it shows *what* accuracy was measured, *that* the classifier was published only after passing R7, *that* the wiring was validated in simulation before enforcement, and *that* the label/action applies only to the right content. Store it with the change record.*

# Troubleshooting — Trainable Classifiers

The failures you will actually hit, with the most likely cause and the fix. Work top-down: **most "the classifier isn't working" reports trace back to seed quality, the classifier not being wired into a control, accuracy tuning, or simulation-vs-enforce confusion — not a portal bug.** Where a portal path or cmdlet parameter is named, **verify it against your tenant** — trainable classifiers and auto-labeling evolve.

## 1. "I published the classifier but nothing happens"

**Symptoms:** The classifier is published, yet no content is being labeled/protected.

**Fix — this is the #1 cause.**

- **A published classifier does nothing on its own.** It must be **wired into a control** —

auto-labeling, DLP, or retention — to have any effect. Build the wiring (Phase 3): `CLSTRAIN-Configure.ps1` creates the auto-labeling policy in simulation.

- **Confirm the classifier is actually referenced** as the rule's condition (`CLSTRAIN-Validate.ps1`

B6 reminds you to verify this in the portal — PowerShell does not always expose the classifier condition).

- **Check the wired policy's mode.** In **simulation**, it records what it *would* label but applies

nothing — that is expected until you enforce (Phase 5). `CLSTRAIN-Validate.ps1` gate **B2** shows the mode.

## 2. The classifier won't train / stays "training"

**Symptoms:** Training does not complete, or takes a very long time.

**Likely causes & fixes**

- **Too few seed samples.** A trainable classifier needs a meaningful minimum of positive examples —

**verify the current minimum/recommended counts for your tenant** (R3). Too few samples stalls or weakens training. Add more representative samples to the seed site and retrain.

- **Seed content not readable/mixed.** The seed SharePoint site must be reachable and contain

documents of a **single** category. Remove off-category items.

- **Processing time.** Training is **not** instant — allow the time Microsoft indicates before

concluding it failed.

## 3. Low accuracy at the Test stage

**Symptoms:** In the **Test** stage the classifier's accuracy is below your R7 threshold.

**Likely causes & fixes**

- **Weak or unrepresentative seed set.** Add more, cleaner, more varied positive examples that reflect the real estate; retrain. Quality of seed = quality of classifier.
- **Test set not truly independent / mislabeled ground truth.** The test items must be **separate** from the seed set and correctly pre-categorized (R4). A wrong ground truth makes a good classifier look bad (or vice-versa).
- **Category too broad or overlapping.** If the category blends with others, the model can't discriminate — narrow the definition or split into multiple classifiers.
- **Do not publish below threshold.** Retrain and retest — never lower the R7 bar to force a publish (Gate A). The tolerance should match impact (audit-only can tolerate more error than an encrypting auto-label).

## 4. The classifier matches the wrong content in the real world (false positives)

**Symptoms:** Test-stage accuracy looked fine, but the **simulation** over the real estate matches documents that are not the target category.

**Likely causes & fixes**

- **Test set didn't reflect production variety.** The real estate has content the test set lacked. Add representative real-world negatives/positives to the seed and retrain.
- **Threshold/impact mismatch.** If the classifier feeds an **encrypting** auto-label, even modest false positives are costly — hold in **simulation**, retrain, and only enforce when real-world precision meets R7 (Gate A4).
- **Tune in simulation** — never by loosening an already-enforcing control.

## 5. A built-in classifier isn't available / behaves differently than expected

**Symptoms:** The built-in classifier you wanted isn't in the list, or matches differently.

**Likely causes & fixes**

- **Catalog varies and evolves.** The set of built-in classifiers depends on licensing and changes over time — **verify the current catalog for your tenant** (R6). If a needed category has no built-in, build a **custom** classifier.

• **Built-in classifiers can't be edited.** You cannot retrain a Microsoft built-in classifier; if its behaviour doesn't fit, build a custom one for that category.

## 6. `New-AutoSensitivityLabelPolicy` / label errors

**Symptoms:** Creating the wiring policy errors on the label, mode, or location.

**Likely causes & fixes**

• **Label not published.** `-ApplySensitivityLabel` must reference a **published** sensitivity label. Confirm the label exists and is published (`CLSTRAIN-Discovery.ps1` inventories labels).

• **Mode/location parameter names evolve.** **Verify the simulation Mode value and the location parameters against `Get-Help New-AutoSensitivityLabelPolicy -Full` for your tenant.** The `CLSTRAIN-Configure.ps1` script prints the plan so you can verify before `-Commit`.

• **Prefer the portal for the first build** if parameter names are uncertain, then read it back with `CLSTRAIN-Discovery.ps1`.

## 7. Can't reference the classifier as a rule condition in PowerShell

**Symptoms:** The rule is created but you can't attach the trainable-classifier condition via cmdlet.

**Fix**

• **The classifier-condition parameter shape evolves and isn't always cleanly scriptable.** Attach the **trainable-classifier condition in the portal** on `AL-Rule-Contracts`, then read it back with `CLSTRAIN-Discovery.ps1`. `CLSTRAIN-Configure.ps1` deliberately creates the rule **shell** and flags that the classifier condition must be attached/verified — it does **not** invent a parameter.

• Run `Get-Help New-AutoSensitivityLabelRule -Full` to check the current condition parameters for your tenant.

## 8. Simulation looks fine but enforcement doesn't label

**Symptoms:** In simulation the classifier matches; after "turning it on," content isn't labeled.

**Likely causes & fixes**

• **Policy still in a Test mode.** Move it out of simulation: portal → turn **On**; PowerShell → `Set-AutoSensitivityLabelPolicy -Identity AL-Classifier-Pilot -Mode Enable`. `CLSTRAIN-Validate.ps1` gate **B2** shows the mode.

• **Processing time.** Auto-labeling at enforce processes content over time — it is **not** instant. Wait, then re-check.

• **Out of scope.** Enforcement applies only to the policy's scope; confirm with

`CLSTRAIN-ScopeHealth.ps1`.

## 9. Wrongly-applied labels need reversing

**Symptoms:** Enforcement applied labels to content that shouldn't have them.

**Fix**

- **Disabling the policy does NOT remove already-applied labels.** `Set-AutoSensitivityLabelPolicy

-Mode Disable` stops *new* labeling but existing labels remain. Plan a controlled **re-label / label change** under change control to reverse them.

- Then fix the root cause (retrain the classifier / tighten the rule) **in simulation** before

re-enforcing — this is exactly why the accuracy gate (R7) and simulation pass exist.

## 10. Activity report is empty or export fails

**Symptoms:** `CLSTRAIN-Report.ps1` returns no activity, or the Activity Explorer export warns/skips.

**Likely causes & fixes**

- **Licensing/permission.** `Export-ActivityExplorerData` is licensing-dependent; the script **warns

and degrades to policy-modes only. **Fall back to** Purview → Activity explorer**, and review the** auto-labeling simulation results** in the portal (that is where would-be-labeled items are shown).

- **Quiet/early window.** Simulation results take time to populate; allow processing time or widen

`-Days`.

## When to escalate to the determination owners (not a technical fix)

Some questions are **not** troubleshooting — they are **determinations** that belong to compliance / records / data owners (see `global-requirements-purview.md` §5):

- Is *this accuracy* good enough to enforce? → the **R7** data owner / approver.
- *Which* label should this category receive? → **compliance / information-protection owners**.
- Is *this content* in scope for classification? → **data owners**.

Provide them the **evidence** (Test-stage accuracy, simulation results, `CLSTRAIN-Report.ps1`, the validation output) — not a verdict.

# Best Practices — Trainable Classifiers

Field-tested guidance for building and running trainable classifiers well. It complements — does not replace — the implementation steps and the pass/fail gates.

## Use built-in classifiers before you build custom ones

• **Check the built-in catalog first.** Microsoft ships pre-trained classifiers (e.g. Resumes, Source Code, and others — verify the current catalog). If one covers your category, use it — no seeding, training, or testing needed.

• **Reserve custom classifiers for organization-specific content** (your contracts, your SOWs, your internal document types) that no built-in classifier recognizes.

## Seed and test like a data scientist

• **Seed with clean, representative, single-category examples.** A classifier is only as good as its seed set. Use enough samples (meet/exceed the Microsoft minimums — verify current counts), all genuinely of the target category, with the variety the real estate contains.

• **Keep the test set separate from the seed set.** Measuring accuracy on the data you trained on is meaningless. The Test stage needs items with a **known ground truth** the model has not seen.

• **Include true negatives in testing.** A classifier that matches *everything* has high recall and useless precision. Test with non-matching items too, and measure precision, not just recall.

• **Retrain iteratively.** If accuracy is low, add/clean seed samples and retrain — do not lower the bar. Classifier quality compounds downstream (every mislabel is a mis-protection).

## Never publish or enforce a classifier that fails its accuracy gate

• **Accuracy tolerance depends on what it drives.** A classifier feeding an **audit-only report** can tolerate more error than one **auto-applying an encrypting label** or a **DLP block**. Agree the threshold (R7) with the data owner *before* publishing, and match tolerance to impact.

• **Sign-off is a determination, not a technical default.** "Good enough" is the data owner's call — capture it in writing.

## The classifier is only half the job — wire it, in simulation

• **Publishing changes nothing on its own.** A classifier has value only once an **auto-labeling / DLP / retention** policy acts on it. Treat the wiring as a first-class deliverable.

• **Always simulate the wired control first.** Auto-labeling can apply **encrypting** labels; DLP can block. Run the control in **simulation** and review what it *would* do across the pilot estate before enforcing. Auto-labeling simulation processes real content — allow time.

• **Reuse one good classifier across controls.** A single published classifier can feed auto-labeling, DLP, and retention — build it once, wire it many times, each simulate-first.

## Mind the high-impact, hard-to-reverse nature of labeling

• **Applied labels don't auto-remove.** Disabling an auto-labeling policy stops *new* labeling but does **not** strip labels already applied. Plan any reversal (re-label / label change) under change control — another reason to get accuracy right before enforce.

• **Start narrow, expand in rings.** Pilot scope first (`SG-Classifier-Pilot`), validate, then widen.

## Operate it, don't set-and-forget

• **Re-check accuracy over time.** Content drifts; a classifier trained a year ago may need retraining as document styles change. Periodically re-test against a fresh ground-truth set.

• **Roll up modes and activity** with `CLSTRAIN-Report.ps1`; review which policies are in simulation vs enforce and what they are matching.

• **Version and name clearly.** Move from `AL-Classifier-Pilot` to a production `AL-Classifier` as you exit pilot; keep a change log of classifier versions and wiring changes.

• **Re-validate after portal changes.** Trainable classifiers and auto-labeling evolve (some settings are newer/preview); re-run `CLSTRAIN-Validate.ps1` after Microsoft feature updates.

## Keep the roles and determinations where they belong

• **Least privilege, split duties.** Classifier management and policy authoring are different roles — do not fold them into one over-privileged account.

• **Determinations stay with the owners.** *Which* label a category deserves, *what* accuracy is acceptable, and *whether* content is in scope are **compliance / data-owner** calls. You provide the classifier, the accuracy evidence, and the wiring, not the judgement (see `global-requirements-purview.md` §5).

## Security & handling of the tooling

• **Read-only /** `-WhatIf` **by default.** Run the provided scripts in their safe default first; only add `-Commit`/`-Apply` when you intend to write, under change control. The scripts **wire** controls; they do **not** invent a cmdlet to create/publish a classifier (that is a portal action).

• **No secrets in code.** The scripts use interactive modern-auth (`Connect-IPPSSession` / `Connect-MgGraph`); never hardcode credentials, tokens, or app secrets.

• **Least-privilege connections.** Connect with the least-privilege role that the task needs, not a standing Global Administrator session.

# Deliverables Summary — Trainable Classifiers in Microsoft Purview

Everything included in this solution, what each piece is for, and how it maps to the deployment model. Use it as the table of contents and the "did we finish?" checklist.

## Documents

| Document | What it gives you |
|---|---|
| **01 — Solution Overview** | What trainable classifiers deliver (classify by meaning, not keywords), the seed→train→test→publish lifecycle, built-in vs custom, the critical **wiring** to labels/DLP/retention, and the Assess→Design→Configure→Pilot→Validate→Expand model. |
| **02 — Requirements** | The solution-specific prerequisites (R1–R8): licensing, classifier + control roles, seed content, a separate test set, the target control/label, the built-in catalog review, the accuracy-acceptance decision, and a pilot scope — each with Verify / Implement-if-missing. References the category `global-requirements-purview.md`. |
| **03 — Implementation Guide** | The ordered build: design (built-in vs custom) → **seed/train/test/publish** the classifier (portal) → **wire** it into an auto-labeling policy **in simulation** → pilot & validate → enforce & expand. Every object named, with a pass/fail gate and rollback per phase. |
| **05 — Testing & Validation** | The pass/fail gates: classifier **accuracy** gates (A), **wiring** configuration gates (B), behavioural tests (C) run in simulation then enforce, and the evidence to capture (D). |
| **06 — Troubleshooting** | The failures you will actually hit — classifier won't train/publish, low accuracy, "published but nothing happens" (not wired), simulation-vs-enforce confusion, classifier condition not scriptable — each with cause and fix. |
| **07 — Best Practices** | How to run it well: built-in before custom, seed/test like a data scientist, never publish a failing classifier, wire-in-simulation, the hard-to-reverse nature of labeling, ongoing operation, and role/determination boundaries. |
| **Executive Summary** | The one-page business-level summary for stakeholders and sign-off. |

## PowerShell tools (5)

All are **read-only or `-WhatIf` by default**, use interactive modern-auth (`Connect-IPPSSession` / `Connect-MgGraph`), and contain **no credentials**. They automate the **wiring** and reporting; they do **not** create/publish classifiers (a portal action) and do **not** invent cmdlets.

| Script | Purpose | Safe default |
|---|---|---|
| **CLSTRAIN-Discovery.ps1** | Inventory existing auto-labeling / DLP / retention policies, sensitivity labels, and where a classifier can be wired — the baseline of what could act on a classifier. | Read-only |
| **CLSTRAIN-Configure.ps1** | Build/update the `AL-Classifier-Pilot` auto-labeling policy + rule **in simulation** (the wiring), applying the target label; requires `-Commit` to write and supports `-WhatIf`. | `-WhatIf` (preview) |
| **CLSTRAIN-Validate.ps1** | Run the wiring configuration gates (policy exists, simulation-first, pilot scope, rule present, target label, classifier referenced) with PASS/FAIL output. | Read-only |
| **CLSTRAIN-Report.ps1** | Roll up auto-labeling/DLP policy modes and, where available, **classification activity** via Activity-explorer export — the operational + evidence view. | Read-only |
| **CLSTRAIN-ScopeHealth.ps1** | Report seed/scope health (pilot-group membership via Graph, policy scope) and preview pilot-scope targeting; `-WhatIf` for any scoping change. Reminds you to confirm classifier status + seed site in the portal. | `-WhatIf` / read-only |

## How it maps to the deployment model

| Phase | Deliverables used |
|---|---|
| **Assess** | 01, 02, `CLSTRAIN-Discovery.ps1`, `CLSTRAIN-ScopeHealth.ps1` |
| **Design** | 02, 03 (classifier map + target label), 07 |
| **Configure (seed/train/test/publish + wire)** | 03 (Phase 2 portal + Phase 3 wiring), `CLSTRAIN-Configure.ps1 -WhatIf` then `-Commit` (simulation) |
| **Pilot (simulation)** | 03, 05 (Gate C simulation pass), `CLSTRAIN-Report.ps1` |

| Phase | Deliverables used |
|---|---|
| **Validate** | 05 (Gate A accuracy + Gate B wiring), `CLSTRAIN-Validate.ps1`, `CLSTRAIN-Report.ps1` |
| **Expand (enforce)** | 03 (Phase 5), 05 (Gate C enforce pass), 06, 07 |

## Definition of done

• Each needed category mapped to a **built-in** or **custom** classifier; custom classifiers **published only after** meeting the **R7 accuracy** threshold (Gate A), signed off by the data owner.

• `AL-Classifier-Pilot` auto-labeling wiring built **in simulation**, all wiring gates (B) PASS, referencing the classifier and applying the target label.

• Behavioural gates (C) PASS in the **simulation pass**, then in the **enforce pass**; matching content labeled, non-matching untouched.

• **Evidence pack** (D) captured — accuracy results, R7 sign-off, wiring validation, simulation results — and stored with the change record.

• Regulatory / policy determinations (which label, what accuracy is acceptable, what is in scope) recorded by the **compliance / data-owner** functions — not by the administrator.

# Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

## CLSTRAIN-Configure.ps1

**Purpose:** Builds the WIRING that acts on a trainable classifier: creates (or updates) an auto-sensitivity-label policy scoped to a pilot, in SIMULATION/TEST mode, that applies a target sensitivity label, plus a rule intended to reference the trainable/built-in classifier as its condition. SAFE BY DEFAULT: previews with -WhatIf and writes NOTHING unless you pass -Commit. It never enforces (never sets Enable) and never creates/publishes a classifier (that is a portal action). HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\CLSTRAIN-Configure.ps1 -WhatIf # preview only (default is preview) .\CLSTRAIN-Configure.ps1 -Commit # create the auto-label policy + rule in SIMULATION .\CLSTRAIN-Configure.ps1 -Commit -PilotGroup "SG-Classifier-Pilot" -LabelName "Confidential" -ClassifierName "TC-Contracts" EXPECTED RESULTS With -Commit: auto-labeling policy 'AL-Classifier-Pilot' exists in a Test/simulation mode, pilot scope, applying the target label, with a rule to host the classifier condition. Validate with CLSTRAIN-Validate.ps1 (expect Gate B PASS). Without -Commit: a printed plan and no changes. .NOTES White-label Microsoft Purview tooling. -WhatIf/preview by default; -Commit gates all writes. No credentials stored. Uses standard New-/Set-AutoSensitivityLabelPolicy and New-/Set-AutoSensitivityLabel Rule; NO invented cmdlets. HOW a trainable classifier is referenced as a rule condition, and the exact simulation Mode value, EVOLVE — the script creates the simulation policy/rule shell and PRINTS that you must attach/verify the classifier condition (Get-Help New-AutoSensitivityLabelRule -Full) or set it in the portal. It does NOT enforce (no Enable) and does NOT create/publish a classifier. > [CmdletBinding()] param( [switch]$Commit, [switch]$WhatIf, [string]$PolicyName = "AL-Classifier-Pilot", [string]$RuleName = "AL-Rule-Contracts", [string]$PilotGroup = "SG-Classifier-Pilot", # pilot scope for the auto-labeling policy [string]$LabelName = "Confidential", # sensitivity label to auto-apply (must be PUBLISHED) [string]$ClassifierName = "TC-Contracts", # PUBLISHED trainable (or built-in) classifier to reference [string]$SimMode = "TestWithoutNotifications" # simulation/test mode; verify the exact value in your tenant )

**How to run:** PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\CLSTRAIN-Configure.ps1 -WhatIf # preview only (default is preview) .\CLSTRAIN-Configure.ps1 -Commit # create the auto-label policy + rule in SIMULATION .\CLSTRAIN-Configure.ps1 -Commit -PilotGroup "SG-Classifier-Pilot" -LabelName "Confidential" -ClassifierName "TC-Contracts" EXPECTED RESULTS With -Commit: auto-labeling policy 'AL-Classifier-Pilot' exists in a Test/simulation mode, pilot scope, applying the target label, with a rule to host the classifier condition. Validate with CLSTRAIN-Validate.ps1 (expect Gate B PASS). Without -Commit: a printed plan and no changes. .NOTES White-label Microsoft Purview tooling. -WhatIf/preview by default; -Commit gates all writes. No credentials stored. Uses standard New-/Set-AutoSensitivityLabelPolicy and New-/Set-AutoSensitivityLabel Rule; NO invented cmdlets. HOW a trainable classifier is referenced as a rule condition, and the exact simulation Mode value, EVOLVE — the script creates the simulation policy/rule shell and PRINTS that you must attach/verify the classifier condition (Get-Help New-AutoSensitivityLabelRule -Full) or set it in the portal. It does NOT enforce (no Enable) and does NOT create/publish a classifier. > [CmdletBinding()] param( [switch]$Commit, [switch]$WhatIf, [string]$PolicyName = "AL-Classifier-Pilot", [string]$RuleName = "AL-Rule-Contracts", [string]$PilotGroup = "SG-Classifier-Pilot", # pilot scope for the auto-labeling policy [string]$LabelName = "Confidential", # sensitivity label to auto-apply (must be PUBLISHED) [string]$ClassifierName =

"TC-Contracts", # PUBLISHED trainable (or built-in) classifier to reference [string]$SimMode = "TestWithoutNotifications" # simulation/test mode; verify the exact value in your tenant )

## CLSTRAIN-Discovery.ps1

**Purpose:** Read-only baseline discovery for trainable classifiers. Because a published classifier only has value once a CONTROL acts on it, this inventories the controls a classifier can be wired into — auto-labeling policies (with mode), DLP policies, and retention policies — plus the sensitivity labels a classifier could apply. This is the "what could act on a classifier" baseline before you build one. Changes nothing. Classifier creation/publish status is confirmed in the Purview portal (there is no supported S&C PowerShell cmdlet to create/publish a trainable classifier). HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\CLSTRAIN-Discovery.ps1 EXPECTED RESULTS A CSV baseline (.\CLSTRAIN-Discovery.csv) of auto-labeling policies (+ Mode), DLP policies, retention policies, and sensitivity labels. Use it to pick the target control + label to wire the classifier into. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell (Get-AutoSensitivityLabelPolicy / Get-DlpCompliancePolicy / Get-RetentionCompliance Policy / Get-Label); none are invented. Trainable classifiers are licensing-dependent and portal-managed — the script WARNS where a control is not readable rather than assuming. > [CmdletBinding()] param( [string]$OutPath = ".\CLSTRAIN-Discovery.csv" )

**How to run:** PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\CLSTRAIN-Discovery.ps1 EXPECTED RESULTS A CSV baseline (.\CLSTRAIN-Discovery.csv) of auto-labeling policies (+ Mode), DLP policies, retention policies, and sensitivity labels. Use it to pick the target control + label to wire the classifier into. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell (Get-AutoSensitivityLabelPolicy / Get-DlpCompliancePolicy / Get-RetentionCompliance Policy / Get-Label); none are invented. Trainable classifiers are licensing-dependent and portal-managed — the script WARNS where a control is not readable rather than assuming. > [CmdletBinding()] param( [string]$OutPath = ".\CLSTRAIN-Discovery.csv" )

## CLSTRAIN-Report.ps1

**Purpose:** Read-only operational + evidence report for trainable classifiers. Summarizes the auto-labeling (and DLP) policies that could act on a classifier and their mode (simulation vs enforce), and — where the tenant/licensing exposes it — exports recent classification/labeling activity from Activity Explorer so you can see what the wired control is matching/applying. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\CLSTRAIN-Report.ps1 .\CLSTRAIN-Report.ps1 -Days 7 EXPECTED RESULTS A console summary + .\CLSTRAIN-PolicyModes.csv (auto-label/DLP policies + mode) and, where available, .\CLSTRAIN-Activity.csv (Activity Explorer classification/labeling events). If activity export is not available in your tenant/licence, the script WARNS and reports policy modes only. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Uses Get-AutoSensitivityLabel Policy / Get-DlpCompliancePolicy / Export-ActivityExplorerData; none invented. Activity Explorer export is licensing-dependent — the script degrades gracefully (warns). Auto-labeling SIMULATION RESULTS themselves are reviewed in the portal. > [CmdletBinding()] param( [int]$Days = 7, [string]$ModesOutPath = ".\CLSTRAIN-PolicyModes.csv", [string]$ActivityOutPath = ".\CLSTRAIN-Activity.csv" )

**How to run:** PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\CLSTRAIN-Report.ps1 .\CLSTRAIN-Report.ps1 -Days 7 EXPECTED RESULTS A console summary + .\CLSTRAIN-PolicyModes.csv (auto-label/DLP policies + mode) and, where available, .\CLSTRAIN-Activity.csv (Activity Explorer classification/labeling events). If activity export is not available in your tenant/licence, the script WARNS and reports policy modes only. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Uses Get-AutoSensitivityLabel Policy / Get-DlpCompliancePolicy / Export-ActivityExplorerData; none invented. Activity Explorer export is licensing-dependent — the script degrades gracefully (warns). Auto-labeling SIMULATION RESULTS themselves are reviewed in the portal. > [CmdletBinding()] param( [int]$Days = 7, [string]$ModesOutPath = ".\CLSTRAIN-PolicyModes.csv", [string]$ActivityOutPath = ".\CLSTRAIN-Activity.csv" )

## CLSTRAIN-ScopeHealth.ps1

**Purpose:** Seed/scope helper for trainable classifiers. Summarizes the pilot user/security group membership from Microsoft Graph and shows the current scope + mode of the auto-labeling wiring policy — so you can see what the wired control WOULD cover before you widen it. Reminds you to confirm the SEED site and the classifier's PUBLISHED status in the portal (neither is readable via S&C PowerShell). Read-only by default; any scope change is preview-only via -WhatIf (this tool does not itself widen scope — it prints the command for you to run under change control). HOW TO RUN PowerShell 5.1+ or 7+ with Microsoft Graph and (optionally) Security & Compliance PowerShell: Connect-MgGraph -Scopes "Group.Read.All" Connect-IPPSSession # optional, to read the policy scope .\CLSTRAIN-ScopeHealth.ps1 .\CLSTRAIN-ScopeHealth.ps1 -PilotGroup "SG-Classifier-Pilot" -WhatIf EXPECTED RESULTS A console summary: pilot-group membership (Graph) and the auto-labeling policy's current scope/mode. With -WhatIf, a PREVIEW of the scope-widening command (nothing is changed). .NOTES White-label Microsoft Purview tooling. Read-only / -WhatIf only. No credentials stored. Uses standard Microsoft Graph (Get-MgGroup / Get-MgGroupMember) and Get-AutoSensitivityLabelPolicy; none invented. The SEED site and classifier PUBLISHED status are confirmed in the Purview portal. > [CmdletBinding()] param( [switch]$WhatIf, [string]$PilotGroup = "SG-Classifier-Pilot", [string]$PolicyName = "AL-Classifier-Pilot", [string]$SeedSite = "Seed-Contracts" )

**How to run:** PowerShell 5.1+ or 7+ with Microsoft Graph and (optionally) Security & Compliance PowerShell: Connect-MgGraph -Scopes "Group.Read.All" Connect-IPPSSession # optional, to read the policy scope .\CLSTRAIN-ScopeHealth.ps1 .\CLSTRAIN-ScopeHealth.ps1 -PilotGroup "SG-Classifier-Pilot" -WhatIf EXPECTED RESULTS A console summary: pilot-group membership (Graph) and the auto-labeling policy's current scope/mode. With -WhatIf, a PREVIEW of the scope-widening command (nothing is changed). .NOTES White-label Microsoft Purview tooling. Read-only / -WhatIf only. No credentials stored. Uses standard Microsoft Graph (Get-MgGroup / Get-MgGroupMember) and Get-AutoSensitivityLabelPolicy; none invented. The SEED site and classifier PUBLISHED status are confirmed in the Purview portal. > [CmdletBinding()] param( [switch]$WhatIf, [string]$PilotGroup = "SG-Classifier-Pilot", [string]$PolicyName = "AL-Classifier-Pilot", [string]$SeedSite = "Seed-Contracts" )

## CLSTRAIN-Validate.ps1

**Purpose:** Read-only validation of the classifier WIRING against the configuration gates in 05-testing.md (Gate B). Confirms the auto-labeling policy exists, is in a Test/simulation mode (during the pilot pass), is scoped to a pilot (not all), applies the intended sensitivity label, and has the expected rule present and enabled. Prints PASS/FAIL per check. Changes nothing. NOTE: the classifier's ACCURACY (Gate A) and whether the rule's condition truly references the classifier are confirmed in the portal — PowerShell does not always expose the trainable-classifier condition. HOW TO RUN PowerShell 5.1+ or 7+ with Security &

Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\CLSTRAIN-Validate.ps1 .\CLSTRAIN-Validate.ps1 -PolicyName "AL-Classifier-Pilot" -LabelName "Confidential" EXPECTED RESULTS A PASS/FAIL line per gate and an overall result. Expect all-PASS in the simulation pass. In the enforce pass, B2 (simulation) is expected to FLIP (policy Mode = Enable) BY DESIGN — that is noted, not an error. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Standard Get-AutoSensitivity Label* cmdlets; none invented. The classifier condition and accuracy are confirmed in the portal. > [CmdletBinding()] param( [string]$PolicyName = "AL-Classifier-Pilot", [string]$RuleName = "AL-Rule-Contracts", [string]$LabelName = "Confidential" )

**How to run:** PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\CLSTRAIN-Validate.ps1 .\CLSTRAIN-Validate.ps1 -PolicyName "AL-Classifier-Pilot" -LabelName "Confidential" EXPECTED RESULTS A PASS/FAIL line per gate and an overall result. Expect all-PASS in the simulation pass. In the enforce pass, B2 (simulation) is expected to FLIP (policy Mode = Enable) BY DESIGN — that is noted, not an error. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Standard Get-AutoSensitivity Label* cmdlets; none invented. The classifier condition and accuracy are confirmed in the portal. > [CmdletBinding()] param( [string]$PolicyName = "AL-Classifier-Pilot", [string]$RuleName = "AL-Rule-Contracts", [string]$LabelName = "Confidential" )